

AI Voice Receptionist

Complete Flow Guide — How Everything Connects

A simple, real-life explanation of your phone AI system: who talks to whom, what each piece does, and what happens when someone calls.

One-line idea

A customer dials your Exotel number → audio streams to your Python bridge → Gemini AI talks live → when needed, n8n does business work (book, log, WhatsApp) → call ends and a summary is saved.

1. Real-life analogy (easiest way to understand)

Think of a busy clinic front desk. Several roles work together:

Role in real life	In your system	Job
Reception phone line (Exotel number)	Exotel (telephony)	Brings the call into the office. Carries voice both ways.
Smart receptionist who listens & speaks	Gemini Live AI	Understands speech, replies naturally, decides when to use tools.
Back-office clerk (files, WhatsApp, sheets)	n8n workflows	Books slots, writes Google Sheet, sends WhatsApp — does not chat on the call.
Switchboard operator connecting everyone	Python voice-agent (bridge.py + app.py)	Converts audio formats, relays streams, hang-up rules, tool calls.
Public front door of the office	Cloudflare tunnel (cloudflared)	Lets Exotel reach your laptop/server on the internet safely.

Example: Priya calls Bright Smile Dental. Exotel is the phone company cable. Gemini is the receptionist on the line. n8n is the person who writes the appointment in the diary and texts the owner on WhatsApp. Your Python app is the headset mixer that keeps everyone in sync.

2. What pieces are used (inventory)

2.1 Telephony — Exotel (primary) / Plivo (fallback)

What: Cloud phone system. Gives you a virtual number people can dial. Supports **bidirectional WebSocket audio** (Voicebot on Exotel, Stream on Plivo).

Why Exotel now: Plivo needs Indian KYC documents (incorporation + tax). Exotel lets you start sooner. Plivo stays as code fallback — flip one env flag later.

Audio formats: Exotel = raw PCM 16-bit mono (usually 8 kHz). Plivo = G.711 mu-law 8 kHz. Your `audio.py` converts between them and what the AI expects.

2.2 AI brain — Gemini Live (now) / OpenAI Realtime (optional)

What: Real-time speech-to-speech model. Caller speaks → AI hears audio → AI speaks audio back. No separate STT + TTS pipeline for the main talk path.

Gemini specifics: Free-tier friendly API key from Google AI Studio. Native audio model. Tuned with short silence detection and `thinking_budget=0` for faster replies.

Tools: AI can call functions (book_appointment, end_call, etc.). Your code runs them and feeds results back so the AI can say “Done, Tuesday at 3 PM.”

2.3 Orchestration — n8n

What: Visual workflow engine. Webhook receives JSON from the bridge. Routes by action: mid-call business actions OR post-call logging.

After call ends: Append row to Google Sheet (voice_calls), optionally send WhatsApp summary via ResilioHub API.

2.4 Your Python voice-agent (the switchboard)

File	Role
app.py	FastAPI: /health, Exotel WS URL, /exotel/stream, /plivo/answer + /plivo/stream
bridge.py	Main call loop: audio relay, barge-in, hang-up watchdog, n8n notify on end
provider_gemini.py	Connects to Gemini Live, sends/receives audio + transcripts + tools
provider_openai.py	Same interface for OpenAI Realtime (swap via AI_PROVIDER)
provider_base.py	Shared events: AudioDelta, SpeechStarted, ToolCall, EndCallRequested
audio.py	mu-law ↔ PCM, resample, RMS speech energy, Exotel FrameBuffer
tools.py	Tool definitions + dispatch to n8n (end_call handled locally)
config.py	Loads .env: keys, timeouts, prompts, providers
plivo_xml.py	XML that tells Plivo to open the bidirectional stream

2.5 Tunnel — cloudflared

Your app runs on localhost:5000. Exotel cannot dial localhost. Cloudflare Tunnel gives a public HTTPS/WSS host. You put that host (no https://) into PUBLIC_HOST.

2.6 Google Sheet + WhatsApp

Sheet tab **voice_calls** columns: call_id, from_phone, ended_at, reason, summary, transcript, duration_sec, business_name. WhatsApp goes to owner number in NOTIFY_WHATSAPP via ResilioHub send-message API.

3. How things are connected (wiring diagram in words)

Read this top → bottom like a phone call journey:

```

Caller phone
  ↓ PSTN / mobile network
Exotel virtual number + Voicebot applet
  ↓ WebSocket (wss://PUBLIC_HOST/exotel/stream)
Cloudflare tunnel → your PC → FastAPI app.py
  ↓ bridge.py opens AI session
Gemini Live (or OpenAI) – live audio in/out
  ↓ when AI needs business action
tools.py → HTTP POST → n8n webhook /voice-agent
  ↓ mid-call: JSON result spoken by AI
  ↓ call end: call_ended → Sheet + WhatsApp
  
```

From	To	Protocol / path	Carries
Exotel	voice-agent	WSS /exotel/stream	PCM audio frames + events
Plivo	voice-agent	HTTPS /plivo/answer then WSS /plivo/stream	XML then mu-law audio
voice-agent	Gemini	SDK Live WebSocket	PCM 16k in / 24k out + tools
voice-agent	n8n	HTTPS POST N8N_WEBHOOK_URL	action JSON + call_ended
n8n	Google Sheets	Google Sheets node	one log row per call
n8n	ResilioHub WA	HTTP API	WhatsApp summary text
cloudflared	Internet	public trycloudflare host	HTTPS + WSS to port 5000

4. Full call flow — second by second (real example)

Scene: Customer Ravi calls ResilioHub WhatsApp CRM number at 11:02 AM. He wants a product demo tomorrow.

T+0s — Dial

Ravi dials Exotel number. Telecom network rings Exotel. Exotel starts your Voicebot call flow.

T+1s — WebSocket open

Exotel connects to `wss://YOUR_HOST/exotel/stream?sample-rate=8000`. Cloudflare forwards to `localhost:5000`. FastAPI accepts WebSocket. `bridge.run(telephony=exotel)` starts.

T+1–2s — AI connect

Bridge creates Gemini provider session with system prompt + greeting instruction. Also starts silence / max-duration watchdog timer.

T+2–4s — Greeting

Gemini speaks: “Thank you for calling ResilioHub... How can I help?” Bridge resamples 24 kHz AI audio → 8 kHz PCM, packs ~100 ms frames, sends media events to Exotel. Ravi hears it.

T+5–20s — Conversation

Ravi talks. Exotel sends media chunks. Bridge converts/resamples to Gemini input rate, updates last-activity time when speech energy (RMS) is heard. Gemini replies in short sentences.

T+20s — Barge-in

If Ravi interrupts while AI talks, Gemini signals interrupted. Bridge sends Exotel c`lear (or Plivo clearAudio) so leftover AI audio is flushed — no talking over the caller.

T+35s — Tool use (optional)

Ravi: “Book a demo tomorrow 3 PM.” AI calls a tool → tools.py POSTs to n8n → n8n returns confirmation → AI says “Booked for tomorrow at 3.”

T+50s — End intent

Ravi: “Thanks, that’s all.” Prompt teaches AI to say a one-line farewell and call end_call with a short summary. Bridge schedules hang-up after END_CALL_GRACE_SEC (~2.5s) so farewell audio can finish.

Or — Silence hang-up

If nobody speaks for SILENCE_TIMEOUT_SEC (e.g. 20s) after AI finished, watchdog ends the call. Also hard stop at MAX_CALL_DURATION_SEC (e.g. 300s).

Teardown — Post-call

Bridge POSTs action=call_ended with transcript snippets, summary, reason, duration, from_phone. n8n: Build Call Log → Append Google Sheet → if NOTIFY_WHATSAPP set → Send WhatsApp Summary. WebSockets close. Done.

5. How each major component is used day-to-day

5.1 Starting the stack (morning checklist)

1. Activate venv, run `python app.py` (port 5000).
2. Run `cloudflared tunnel --url http://localhost:5000`.
3. Copy new tunnel host into `PUBLIC_HOST` if it changed; restart app.
4. Confirm `https://PUBLIC_HOST/health` returns status ok.
5. n8n workflow active; Exotel Voicebot pointing at your WSS URL.

5.2 Config knobs that change behavior

Setting	Effect in real life
<code>AI_PROVIDER / TELEPHONY_PROVIDER</code>	Which brain / which phone company
<code>GEMINI_SILENCE_MS (~500)</code>	How long after you stop talking before AI answers — lower = snappier
<code>GEMINI_THINKING_BUDGET=0</code>	Less “thinking delay” before speech
<code>SILENCE_TIMEOUT_SEC</code>	Auto hang-up if caller goes quiet
<code>MAX_CALL_DURATION_SEC</code>	Hard max call length
<code>END_CALL_GRACE_SEC</code>	Time for goodbye audio to finish
<code>SYSTEM_PROMPT / GREETING</code>	Personality + when to end_call
<code>N8N_WEBHOOK_URL</code>	Where business actions + call logs go
<code>NOTIFY_WHATSAPP</code>	Owner phone for summary WhatsApp

5.3 Mid-call tools vs post-call automation

Mid-call: AI still on the phone. Tool result must come back quickly so AI can speak it. Example: `check_availability` → “We have 3 PM free.”

Post-call (call_ended): Caller already hung up. No need to speak. Write sheet + WhatsApp. Example: Owner gets “Ravi wanted demo tomorrow 3 PM; call 4m12s.”

end_call tool: Handled inside Python (does not need n8n). It only tells the bridge: hang up soon, here is the summary string to include in `call_ended`.

6. Audio path explained simply

Phones and AIs do not speak the same “audio language.” The bridge is a translator.

Direction	What happens
Caller → AI (Exotel)	PCM 8 kHz from Exotel → resample to Gemini input (16 kHz) → <code>send_realtime_input</code>
AI → Caller (Exotel)	Gemini PCM 24 kHz → resample to 8 kHz → <code>FrameBuffer</code> (~3200 bytes) → Exotel media
Caller → AI (Plivo)	mu-law 8 kHz → PCM → then to AI provider format
Barge-in	Stop playing leftover AI audio: Exotel event <code>clear</code> / Plivo <code>clearAudio</code>

Kitchen analogy: Exotel sends chopped tomatoes (8 kHz PCM). Gemini wants puree (16 kHz). Gemini returns soup (24 kHz). You re-chop to tomato size before serving the caller. `FrameBuffer` = don’t send half a spoonful — Exotel wants full spoonfuls (~100 ms).

7. n8n workflow roles

Imported from `n8n/voice_agent_actions.json`:

- Webhook trigger receives every POST from voice-agent.
- **Is Call Ended?** — if yes → Build Call Log → Google Sheet → maybe WhatsApp.
- If not ended → Handle Action (book, lead, FAO, etc.) → return JSON for AI to speak.

There is no “AI chat node” inside n8n for the live call. The live AI is Gemini/OpenAI in Python. n8n is the back office only. That is why you do not see an AI conversation node in the call path.

8. Swapping providers later (no rewrite)

Architecture uses a provider interface. Same bridge code.

- Want OpenAI voice: set `AI_PROVIDER=openai` + API key.
- Want Plivo after KYC: set `TELEPHONY_PROVIDER=plivo`. Answer URL = `https://PUBLIC_HOST/plivo/answer`.

Example: Like changing SIM cards in the same phone — the “phone” (bridge) stays; only the network (Exotel/Plivo) or the brain (Gemini/OpenAI) changes.

9. End-to-end story cards (practice understanding)

Story A — Quick FAQ, then goodbye

Caller asks clinic hours. AI answers from prompt (no tool). Caller says thanks. AI farewell + end_call. Sheet row: reason=thanks, short summary. Owner WhatsApp optional.

Story B — Book appointment

Caller wants cleaning Tuesday. AI calls book tool → n8n checks/writes calendar or CRM → returns confirmation → AI speaks it → caller bye → call_ended logs transcript.

Story C — Ghost silence

Caller puts phone down. After ~20s silence (configurable), watchdog hangs up. Summary may say silence timeout. Still logged to sheet.

Story D — Long monologue problem (why prompts matter)

If SYSTEM_PROMPT allows long speeches, Ravi waits forever for AI to finish. That is why prompt says: max 2 short sentences, one question at a time, thinking_budget=0.

10. Quick mental map (memorize this)

Phone company moves sound. **AI** understands & talks. **Python bridge** translates & enforces hang-up. **n8n** does paperwork. **Cloudflare** opens the door to your machine. **Sheet/WhatsApp** remember what happened.

If this breaks...	Check this first
Exotel can't connect	cloudflared running? PUBLIC_HOST match? /health OK? Voicebot WSS URL?
AI silent / slow	GEMINI_API_KEY, model name, SILENCE_MS, SYSTEM_PROMPT length
Call never ends	end_call in prompt? SILENCE_TIMEOUT_SEC? bridge logs hangup?
No sheet / WhatsApp	n8n active? action=call_ended path? Sheet credentials? NOTIFY_WHATSAPP?
Audio choppy	EXOTEL_SAMPLE_RATE match? FrameBuffer / network lag / tunnel

Document generated for the voice-agent project. Stack: Exotel + Gemini Live + FastAPI bridge + n8n + Google Sheets + WhatsApp (ResilioHub) + cloudflared.